

Nama : M. Davana Yudista Firdausy

Nrp : 3125522002

Kelas : A D3 TI-SM

BAGIAN 1: IDENTITAS PROYEK, SPRINT PLAN & AUDIT BEHAVIOR

1. Inisiasi Proyek

- **Nama Proyek:** Sistem Manajemen Inventaris Toko (Iterasi Pengembangan Sistem)
- **Product Goal:** Saya mengembangkan sistem inventaris toko yang solid, di mana entitas-entitas dalam sistem dapat merespons pemanggilan operasional dengan perilaku spesifik masing-masing tanpa merusak struktur hierarki yang telah saya bangun sebelumnya.
- **Sprint Goal:** Saya menerapkan konsep polymorphism agar subclass dapat merespons pemanggilan method yang sama dengan perilaku berbeda melalui method overriding dan dynamic binding, serta mengimplementasikan variasi pemanggilan lewat method overloading.

2. Sprint Backlog

- **SB-01:** Mengaudit superclass dan subclass sebelumnya untuk menemukan kandidat polymorphism (DONE)
- **SB-02:** Menentukan behavior umum (prosesTugas()) yang dapat di-override (DONE)
- **SB-03:** Mengimplementasikan method overriding pada Petugas dan Pemasok (DONE)
- **SB-04:** Mengimplementasikan method overloading pada perhitungan total transaksi (DONE)
- **SB-05:** Mengimplementasikan konsep upcasting menggunakan tipe referensi Person (DONE)
- **SB-06:** Membuat polymorphic collection (Person[]) untuk iterasi objek heterogen (DONE)
- **SB-07:** Melakukan pengujian dynamic binding dan robustness (DONE)

3. Hierarchy Class & Penentuan Behavior Polymorphic

Hierarki sistem yang saya buat terdiri dari Superclass Person dan Subclass Petugas serta Pemasok.

- **Behavior untuk Overriding:** Method prosesTugas() dan tampilkanProfil(). Di superclass Person, prosesTugas() hanya mencetak tugas umum. Di subclass, method ini saya override agar mencetak tugas spesifik: Petugas melayani pelanggan, sedangkan Pemasok menyiapkan manifes distribusi.

BAGIAN 2: CLASS DIAGRAM, OVERRIDING, OVERLOADING & UPCASTING

1. Class Diagram Terbaru

Rancangan class diagram mencakup hubungan pewarisan antara Person sebagai superclass terhadap Petugas dan Pemasok, serta keterkaitan dengan komponen Transaksi dan Gudang.

2. Analisis Polymorphism dalam Proyek Saya

- **Method Overriding (Runtime Polymorphism):** Saya menerapkannya ketika subclass Petugas dan Pemasok menyediakan implementasi spesifik untuk prosesTugas() menggunakan anotasi @Override. Ini mengubah behavior warisan menjadi aksi yang kontekstual sesuai peran masing-masing aktor.
- **Method Overloading (Compile-time Polymorphism):** Saya terapkan pada class Transaksi melalui method hitungTotalBerdasarkanSubtotal. Ada dua versi yang saya buat: satu tanpa parameter (total normal) dan satu dengan parameter double persentaseDiskon. Compiler Java menentukan method yang dieksekusi saat kompilasi (Static Binding) berdasarkan signature parameternya.
- **Upcasting & Dynamic Method Dispatch:** Saya melakukan upcasting dengan menyimpan objek Petugas atau Pemasok ke dalam referensi bertipe Person (Person p = new Petugas(...)). Saat p.prosesTugas() dipanggil, Java Virtual Machine (JVM) secara dinamis mengenali tipe objek aktual di memori dan mengarahkannya ke method Petugas.prosesTugas().

BAGIAN 3: PENGUJIAN, REVIEW & EVALUASI

1. Pembuktian Pengujian melalui Tangkapan Layar

- **Test Case 1 (Method Overloading):** Sistem yang saya buat berhasil memproses pengeluaran stok secara real-time dan mencetak detail transaksi. Pemanggilan hitungTotalBerdasarkanSubtotal() tanpa argumen menghasilkan

Rp490.000,00, sedangkan dengan argumen diskon 10.0% menghasilkan Rp441.000,00. Ini membuktikan Compile-time Polymorphism berjalan lancar.

- **Test Case 2 (Upcasting & Dynamic Dispatch):** Objek Petugas dan Pemasok saya-upcast ke referensi Person (p1 dan p2). Saat prosesTugas() dipanggil, output yang muncul spesifik sesuai peran masing-masing aktor, membuktikan terjadinya Runtime Polymorphism.
- **Test Case 3 (Polymorphic Collection):** Array Person[] saya isi dengan elemen heterogen (2 Petugas dan 2 Pemasok). Menggunakan enhanced for-loop, setiap elemen menjalankan method tampilkanProfil() dan prosesTugas() secara tepat tanpa memerlukan type-casting manual (instanceof / downcasting).
- **Test Case 4 (Negative Testing & Robustness):** Saya menguji overloaded method dengan diskon negatif (-15%) yang ditolak sehingga total tetap normal, serta diskon berlebih (120%) yang otomatis dibatasi maksimal 100% (gratis). Pengujian ditutup dengan pencetakan agregasi Gudang dan status berhasil 100%.

2. Tabel Pengujian Dynamic Binding

Reference Type	Object Aktual	Method Dipanggil	Output Eksekusi
Person (p0)	Person	prosesTugas()	[Person] Melakukan tugas umum.
Person (p1)	Petugas	prosesTugas()	[Petugas] Melayani pelanggan dan memverifikasi stok gudang.
Person (p2)	Pemasok	prosesTugas()	[Pemasok] Menyiapkan manifes dan mendistribusikan barang ke gudang.

3. Evaluasi & Review Pengerjaan

Seluruh rancangan mulai dari superclass, subclass, overriding, overloading, upcasting, polymorphic collection, hingga dynamic binding telah berhasil saya implementasikan dan diuji dengan hasil program berjalan tanpa kendala (100% sukses).

4. Retrospective & Catatan Pengembangan

- **What Went Well?:** Pemisahan behavior polimorfik melalui method overriding berhasil saya jalankan di dalam koleksi Person[] tanpa perlu menggunakan seleksi kondisi (if-else/switch) yang kompleks.

- **What Went Wrong?:** Tidak ada kendala berarti, meskipun saya perlu ketelitian ekstra dalam membedakan static binding pada overloading dan dynamic binding pada overriding.
- **Improvement:** Untuk pengembangan berikutnya, superclass Person dan method prosesTugas() bisa saya sempurnakan menjadi Abstract Class dan Abstract Method, karena objek Person murni seharusnya tidak diinstansiasi langsung dalam konteks aplikasi nyata.

```
PS C:\Users\dava1\Documents\KULIAH\TUGAS KULIAH SEMESTER 3\PEMROGRAMAN BERBASIS OBJEK\CODE> java -cp "P6_3125522002_M Davan
a Yudista Firdausy" src.Main
=====
                PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK - MODUL 6
                POLYMORPHISM, METHOD OVERRIDING, METHOD OVERLOADING, & DYNAMIC BINDING
=====
Nama Mahasiswa : M. Davana Yudista Firdausy
NRP            : 3125522002
Kelas         : A D3 TI-SM
Institusi      : PENS PSDKU Sumenep
Proyek         : Sistem Manajemen Inventaris Toko (Iterasi P5 -> P6)
=====

-> [Superclass Constructor] Person dieksekusi untuk ID: PTG-002
-> [Subclass Constructor] Petugas dieksekusi untuk Jabatan: Kasir Senior

#####
### TEST CASE 1: METHOD OVERLOADING (COMPILE-TIME POLYMORPHISM) ###
#####

[Langkah 1] Membuat Objek Transaksi dan Mengisi Item Detail Transaksi:
Berhasil [Barang]: Mengurangi 10 unit dari [BRG-001] Buku Tulis Hardcover (Sisa stok: 90)
Sukses [Transaksi TRX-OUT-2026-001]: Berhasil mencatat 10 unit [Buku Tulis Hardcover] ke dalam detail transaksi.
Berhasil [Barang]: Mengurangi 20 unit dari [BRG-002] Pulpen Gel 0.5mm (Sisa stok: 180)
Sukses [Transaksi TRX-OUT-2026-001]: Berhasil mencatat 20 unit [Pulpen Gel 0.5mm] ke dalam detail transaksi.
Berhasil [Barang]: Mengurangi 5 unit dari [BRG-003] Kertas HVS A4 70gr (Sisa stok: 45)
Sukses [Transaksi TRX-OUT-2026-001]: Berhasil mencatat 5 unit [Kertas HVS A4 70gr] ke dalam detail transaksi.
```

```
[Langkah 2] Menampilkan Rincian Transaksi:
=====
                DETAIL TRANSAKSI TOKO
=====
ID Transaksi   : TRX-OUT-2026-001
Jenis Transaksi : Barang Keluar
Petugas PIC    : PTG-002 - Siti Nurhaliza (Kasir Senior | Shift: Siang)
=====
Rincian Item Transaksi (Komposisi):
* [BRG-001 ] Buku Tulis Hardcover      | Qty: 10 | @Rp 15.000,00 | Subtotal: Rp 150.000,00
* [BRG-002 ] Pulpen Gel 0.5mm          | Qty: 20 | @Rp 5.000,00  | Subtotal: Rp 100.000,00
* [BRG-003 ] Kertas HVS A4 70gr       | Qty: 5  | @Rp 48.000,00 | Subtotal: Rp 240.000,00
=====
Total Macam Item : 3 SKU
Total Subtotal   : Rp490.000,00
=====

[Langkah 3] Pengujian Overloaded Methods pada Objek Transaksi:
* Pemanggilan Method 1 - hitungTotalBerdasarkanSubtotal() : Rp490.000,00
* Pemanggilan Method 2 - hitungTotalBerdasarkanSubtotal(10,0%) : Rp441.000,00 (Hemat: Rp49.000,00)
* Pemanggilan Method 2 - hitungTotalBerdasarkanSubtotal(25,0%) : Rp367.500,00 (Hemat: Rp122.500,00)

Penjelasan Konsep Test Case 1:
-> Method Overloading terjadi di dalam class yang sama (Transaksi).
-> Compiler Java menentukan secara pasti method mana yang dieksekusi berdasarkan parameter signature pada saat proses kompilasi (Static Binding / Early Binding).
```

```
#####
### TEST CASE 2: UPCASTING & DYNAMIC METHOD DISPATCH ###
#####

[Langkah 1] Inisialisasi Objek dengan Upcasting ke Superclass Person:
-> Upcasting Petugas ke referensi Person (Person p1 = new Petugas(...)):
  -> [Superclass Constructor] Person dieksekusi untuk ID: PTG-001
  -> [Subclass Constructor] Petugas dieksekusi untuk Jabatan: Supervisor Gudang
-> Upcasting Pemasok ke referensi Person (Person p2 = new Pemasok(...)):
  -> [Superclass Constructor] Person dieksekusi untuk ID: PMS-101
  -> [Subclass Constructor] Pemasok dieksekusi untuk Perusahaan: PT Gramedia Mitra Sarana
-> Membuat Objek Murni Superclass Person untuk Kontras (Person p0 = new Person(...)):
  -> [Superclass Constructor] Person dieksekusi untuk ID: PRS-000

[Langkah 2] Memanggil prosesTugas() pada Seluruh Referensi Person:
1. Referensi p0 (Tipe Referensi: Person, Tipe Objek: Person):
  Output: [Person] Melakukan tugas umum.
2. Referensi p1 (Tipe Referensi: Person, Tipe Objek: Petugas):
  Output: [Petugas] Melayani pelanggan dan memverifikasi stok gudang.
3. Referensi p2 (Tipe Referensi: Person, Tipe Objek: Pemasok):
  Output: [Pemasok] Menyiapkan manifes dan mendistribusikan barang ke gudang.

Penjelasan Konsep Test Case 2:
-> p1 dan p2 sama-sama dideklarasikan bertipe Person (Upcasting).
-> Namun saat p1.prosesTugas() dipanggil, JVM secara dinamis mengarahkan eksekusi ke method Petugas.prosesTugas().
-> Begitu juga dengan p2.prosesTugas(), JVM mengarahkan eksekusi ke Pemasok.prosesTugas().
-> Inilah yang membuktikan Dynamic Method Dispatch (Runtime Polymorphism).
```

```
#####
### TEST CASE 3: POLYMORPHIC COLLECTION & DYNAMIC BINDING ###
#####

[Langkah 1] Mempersiapkan Koleksi Polimorfik (Person[]):
-> [Superclass Constructor] Person dieksekusi untuk ID: PTG-001
-> [Subclass Constructor] Petugas dieksekusi untuk Jabatan: Supervisor Gudang
-> [Superclass Constructor] Person dieksekusi untuk ID: PTG-002
-> [Subclass Constructor] Petugas dieksekusi untuk Jabatan: Kasir Senior
-> [Superclass Constructor] Person dieksekusi untuk ID: PMS-101
-> [Subclass Constructor] Pemasok dieksekusi untuk Perusahaan: PT Gramedia Mitra Sarana
-> [Superclass Constructor] Person dieksekusi untuk ID: PMS-102
-> [Subclass Constructor] Pemasok dieksekusi untuk Perusahaan: CV Sinar Grafika
Koleksi daftarAktor (Person[]) berhasil diisi dengan 4 elemen heterogen:
[0] Petugas (Ahmad Fauzi)
[1] Petugas (Siti Nurhaliza)
[2] Pemasok (PT Gramedia Mitra Sarana)
[3] Pemasok (CV Sinar Grafika)
```

```
[Langkah 2] Iterasi Koleksi Menggunakan Enhanced For-Loop:
Setiap elemen dipanggil method tampilkanProfil() dan prosesTugas():

-----
Iterasi Elemen #1 | Tipe Objek Aktual: Petugas (Referensi: Person)
-----
=== Profil Petugas Toko ===
ID          : PTG-001
Nama        : Ahmad Fauzi
Alamat      : Jl. Trunojoyo No. 12, Sumenep
Jabatan     : Supervisor Gudang
Shift Kerja : Pagi
-----
Tugas Aktor : [Petugas] Melayani pelanggan dan memverifikasi stok gudang.

-----
Iterasi Elemen #2 | Tipe Objek Aktual: Petugas (Referensi: Person)
-----
=== Profil Petugas Toko ===
ID          : PTG-002
Nama        : Siti Nurhaliza
Alamat      : Jl. Diponegoro No. 45, Sumenep
Jabatan     : Kasir Senior
Shift Kerja : Siang
-----
Tugas Aktor : [Petugas] Melayani pelanggan dan memverifikasi stok gudang.
```

```
-----
Iterasi Elemen #3 | Tipe Objek Aktual: Pemasok (Referensi: Person)
-----

=== Profil Pemasok Inventaris ===
ID      : PMS-101
Nama    : Budi Santoso
Alamat  : Jl. Palmerah Barat No. 29, Jakarta
Perusahaan : PT Gramedia Mitra Sarana
Kategori : Alat Tulis & Buku
-----

Tugas Aktor : [Pemasok] Menyiapkan manifes dan mendistribusikan barang ke gudang.

-----

Iterasi Elemen #4 | Tipe Objek Aktual: Pemasok (Referensi: Person)
-----

=== Profil Pemasok Inventaris ===
ID      : PMS-102
Nama    : Dewi Lestari
Alamat  : Kawasan Industri Sier No. 8, Surabaya
Perusahaan : CV Sinar Grafika
Kategori : Kertas & Cetakan
-----

Tugas Aktor : [Pemasok] Menyiapkan manifes dan mendistribusikan barang ke gudang.

Penjelasan Konsep Test Case 3:
-> Seluruh elemen dalam loop bertipe referensi sama, yaitu 'Person aktor'.
-> Saat runtime, JVM melakukan Dynamic Method Lookup (Virtual Method Table / vtable).
-> Petugas mengeksekusi format profil Petugas dan tugas spesifik Petugas.
-> Pemasok mengeksekusi format profil Pemasok dan tugas spesifik Pemasok.
-> Tidak diperlukan type-casting manual (instanceof / downcasting) untuk menjalankan perilaku spesifik!
```

```
#####
### TEST CASE 4: NEGATIVE TESTING & ROBUSTNESS METHOD OVERLOADING ###
#####

[Langkah 1] Pengujian Nilai Diskon Negatif (-15%):
    Total transaksi setelah diskon -15%: Rp490.000,00 (Total normal tetap terjaga)

[Langkah 2] Pengujian Nilai Diskon Berlebih (120%):
    Peringatan [Transaksi]: Persentase diskon (120.0%) melebihi 100%! Maksimal diskon 100% diterapkan.
    Total transaksi setelah diskon 120%: Rp0,00 (Maksimal diskon 100% = gratis)

[Langkah 3] Pengujian Agregasi Gudang & Pengurangan Stok Real-Time:
    Sukses [Gudang Gudang Distribusi Sumenep]: Agregasi barang [BRG-001] Buku Tulis Hardcover berhasil didaftarkan.
    Sukses [Gudang Gudang Distribusi Sumenep]: Agregasi barang [BRG-002] Pulpen Gel 0.5mm berhasil didaftarkan.
    Sukses [Gudang Gudang Distribusi Sumenep]: Agregasi barang [BRG-003] Kertas HVS A4 70gr berhasil didaftarkan.
```

```
=====
                        INVENTARIS GUDANG (AGREGASI OBJEK)
=====
Kode Gudang      : GDG-SMP-01
Nama Gudang      : Gudang Distribusi Sumenep
Jumlah Jenis Brg: 3 SKU
-----
KODE      | NAMA BARANG      | KATEGORI      | STOK      | HARGA SATUAN      | TOTAL NILAI
-----
0 | Rp  900.000,00
  BRG-003 | Kertas HVS A4 70gr      | Kertas      | 45      | Rp  48.000,00 | Rp2.160.000,00
-----
Total Unit Stok : 315 unit
Total Nilai Aset : Rp4.410.000,00
=====

=====
                        SELURUH PENGUJIAN MODUL 6: POLYMORPHISM BERHASIL 100% TANPA KENDALA!
=====
PS C:\Users\dava1\Documents\KULIAH\TUGAS KULIAH SEMESTER 3\PEMROGRAMAN BERBASIS OBJEK\CODE> █
```